

How to Run EasyCart 2.1 Locally

This guide explains how to start the development environment for both the **Spring Boot Backend** and the **Vanilla JS Frontend**.

🚀 Quick Start Commands

Open two separate terminals in your project root:

Terminal 1: The Backend (Spring Boot)

```
```powershell
cd backend
mvn spring-boot:run
```
```

Wait for the log: `Started EasyCartSmeApplication in ... seconds`

Terminal 2: The Frontend (Static Server)

```
```powershell
cd frontend
npx -y http-server -p 3000
```
```

Your website will be live at `http://localhost:3000`

📦 Understanding Maven in Spring Boot

Since you mentioned remembering Maven, here is a breakdown of why we use it and what it does for your Java application.

1. What is Maven?

Maven is a build automation and project management tool. In the Java world, it is the standard "manager" for everything your project needs to function.

2. Key Responsibilities

- * **Dependency Management**: Instead of manually downloading `.jar` files (libraries) like Spring Security or PostgreSQL drivers, you list them in a file called `pom.xml`. Maven automatically downloads them from the "Maven Central" repository.
- * **Standardized Structure**: Maven enforces a specific folder layout (`src/main/java`, `src/main/resources`, etc.). This makes it easy for any Java developer to understand the project instantly.
- * **Build Lifecycle**: Maven handles the steps to turn your code into a running app:
 1. **Compile**: Converts `.java` files to `.class` files.
 2. **Test**: Runs your unit tests.
 3. **Package**: Bundles everything into a single file (like a `.jar`) that can be deployed to a server.
 4. **Run**: The command `mvn spring-boot:run` specifically tells Maven to compile the code and then use the Spring Boot plugin to launch the application.

3. The `pom.xml` File

Think of the `pom.xml` (Project Object Model) as the **identity card** of your project. It contains:

- * Project version and name.
- * The list of dependencies (libraries).
- * Plugins (like the Spring Boot plugin) that add extra features to the build process.

🛠 Troubleshooting Tips

* **Port Conflicts**: If port 8080 or 3000 is already in use, the servers will fail to start. You can stop the previous processes or change the ports in `application.properties` (for backend) or the `-p` flag (for frontend).

* **CORS Issues**: The backend is configured to allow requests from `http://localhost:3000`. If you run the frontend on a different port, you must update `cors.allowed-origins` in the backend's `application.properties`.